Image Fragment Reconstruction: A Siamese Network Approach via Adjacency Prediction

May 1, 2026

Contents

1 Core Idea

The central observation is that two fragments cut from the same image and placed next to each other in the grid share a common boundary — colours bleed over, textures continue, and objects span tiles. This visual continuity is a strong and reliable signal that does not require any human annotation to exploit: since the experimenter creates the grid, the ground-truth adjacency of every fragment pair is known at all times.

The proposed approach uses adjacency prediction as a **pretext task** (Noroozi and Favaro 2016). The model is trained to solve a surrogate problem — predicting whether two fragments are spatially adjacent — and in doing so is forced to learn visual representations that capture source identity as a side effect. At inference time the pretext objective is discarded and the learned representations are used for clustering.

This is self-supervised throughout: no labels beyond those derivable from the grid structure are ever used.

2 Why Adjacency Prediction Helps

The naive contrastive approach (Section 3 of the companion document) faces a fundamental tension: intra-image fragments can be visually dissimilar (a sky tile vs. a ground tile from the same photograph), while inter-image fragments can be visually similar (two sky tiles from different photographs). Adjacency prediction addresses this directly:

- To predict whether two fragments are adjacent, the model must first determine whether they could plausibly come from the same image at all — source identity is a necessary precondition for adjacency.
- Adjacency exploits **local edge continuity** — a signal that is orthogonal to global visual similarity and therefore less susceptible to the sky-tile problem.
- The pretext task is harder than simple contrastive learning, which forces the model to learn richer representations.

3 Architecture: Siamese Network

The model consists of two components:

Shared CNN encoder. A single convolutional neural network maps each 16×16×3 fragment to a fixed-dimensional embedding vector. The encoder is **shared** — the same weights are used for both fragments in a pair. This is the defining property of a Siamese network (Bromley et al. 1994): weight sharing ensures that the comparison is symmetric and that the encoder learns a universal fragment representation rather than one tuned to a specific input slot.

Comparison head (MLP). A small multi-layer perceptron (a fully connected network with one or two hidden layers) takes as input the two embeddings — concatenated, or combined via their difference or element-wise product — and outputs a single scalar in $[0, 1]$: the predicted probability that the two fragments are spatially adjacent.

The full forward pass for a pair (f_i, f_j) is:

$$p_{ij} = \mathrm{MLP}\big(\mathrm{CNN}(f_i) \,\|\, \mathrm{CNN}(f_j)\big)$$

where $\|$ denotes concatenation.

4 Training

4.1 Constructing Training Labels

From a sample of 10 images cut into a 4×4 grid, each fragment has at most 4 adjacent neighbours (fewer at corners and edges). For a sample of 160 fragments the total number of pairs is:

$$\binom{160}{2} = 12{,}720$$

Each pair receives a binary label: $y_{ij} = 1$ if the two fragments are adjacent within the same image, $y_{ij} = 0$ otherwise. This label is derived entirely from the known grid positions and requires no human annotation.

4.2 Class Imbalance

Adjacent pairs are rare relative to non-adjacent pairs. Within one image of 16 fragments there are 24 adjacent pairs and $\binom{16}{2} - 24 = 96$ non-adjacent same-image pairs, plus all cross-image pairs. Across the full 160-fragment sample the positive rate is low. This imbalance should be addressed via weighted loss or by subsampling negatives during training.

4.3 Loss Function

Binary cross-entropy is the natural loss for a binary prediction task:

$$\mathcal{L} = -\frac{1}{N} \sum_{(i,j)} \Big[y_{ij} \log p_{ij} + (1 - y_{ij}) \log(1 - p_{ij}) \Big]$$

4.4 Computational Cost

The CNN encoder runs once per fragment (160 forward passes per sample). All pairwise embeddings are then combined using tensor operations, and the lightweight MLP processes all 12,720 pairs. Since the expensive step (the CNN) runs only 160 times, the computational overhead is manageable.

5 From Pretext Task to Clustering

After training, the model produces a predicted probability p_{ij} for every pair of fragments. These probabilities define a **fully connected weighted graph** over the 160 fragments, where edge weight $w_{ij} = p_{ij}$ reflects the model's confidence that fragments i and j are adjacent — and by extension, that they originate from the same image.

Decision point: clustering on the graph. **Spectral clustering** is the natural choice for a weighted graph. It uses the eigenstructure of the graph Laplacian to find clusters that are internally well-connected and externally weakly connected. With $k = 10$ known clusters, the k smallest eigenvectors of the Laplacian are used.

This is more principled than applying k-means directly to embeddings, because the graph structure captures the full relational information between all fragment pairs rather than just the position of individual embeddings in space.

Metric consistency. Unlike the contrastive approach, there is no distance metric to align between training and clustering — the edge weights are probabilities produced directly by the model, and spectral clustering operates on those weights natively.

6 Evaluation Metric: Adjusted Rand Index

The primary evaluation metric is the **Adjusted Rand Index (ARI)**. It measures how well the predicted clusters match the true groupings by counting fragment pairs:

- A pair is a **true positive** if both fragments are in the same true group and the same predicted cluster.
- A pair is a **true negative** if they are in different true groups and different predicted clusters.
- All other pairs are errors.

The "adjusted" part corrects for chance — a random clustering would score ARI = 0, a perfect clustering scores ARI = 1, and a clustering worse than random can score below zero.

ARI is preferred over simpler metrics such as purity because purity only looks at the dominant class in each cluster and ignores how errors are distributed. For example, a cluster containing 60% of fragments from image A and 40% from image B scores the same purity as a cluster containing 60% from image A, 20% from image B, and 20% from image C — despite the latter being more disordered. ARI captures this distinction.

NMI (Normalised Mutual Information) is reported alongside ARI as a complementary metric. It measures how much information the predicted clusters share with the true groupings, normalised to $[0, 1]$.

7 Decision Points Summary

Decision	Choice	Alternatives
Training objective	Adjacency prediction (pretext task)	Contrastive loss
Architecture	Siamese CNN + MLP head	Single encoder
Embedding combination	Concatenation	Difference, element-wise product
Loss function	Binary cross-entropy	Focal loss (for imbalance)
Class imbalance	Weighted loss or negative subsampling	None
Clustering algorithm	Spectral clustering	k-means on embeddings
Number of clusters	$k = 10$ (known)	Discovered automatically
Evaluation metric	ARI, NMI	Purity

8 Limitations

- The augmentation in `data.py` applies rotation and shift to the full image before cutting. This distorts the boundaries between fragments, weakening the edge continuity signal that adjacency prediction relies on.
- Adjacency is a local signal — it says nothing directly about fragments that are far apart within the same image. The model must generalise from local adjacency to global source identity.
- The graph has 12,720 edges. For larger grids or larger samples this grows quadratically and may become a bottleneck.

TODO

- **Dropout** — randomly disable neurons during training to prevent the model from relying on memorised patterns. Add as a hyperparameter in `config.yaml`.
- **Weight decay** — penalise large weights via L2 regularisation, discouraging memorisation. Expose as a hyperparameter in `config.yaml`.
- **Limit model capacity** — a small CNN has less capacity to memorise than a large one. Justify the chosen architecture size explicitly in the report.
- **Balanced spectral clustering** — the current spectral clustering implementation does not enforce equal cluster sizes. Since each source image always produces exactly 16 fragments, the true clusters are perfectly balanced. Constraining the clustering to produce balanced clusters of 16 would reduce ambiguity and likely improve ARI. This requires a balanced variant of spectral clustering or a post-processing step that reassigns fragments to enforce balance.

References

Bromley, Jane et al. (1994). "Signature Verification Using a Siamese Time Delay Neural Network". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 6, pp. 737–744.

Noroozi, Mehdi and Paolo Favaro (2016). "Unsupervised Learning of Visual Representations by Solving Jigsaw Puzzles". In: *European Conference on Computer Vision (ECCV)*. Springer, pp. 69–84.